

Instituto Politécnico de Bragança
Escola Superior de Tecnologia e de Gestão

Business Incubator Management Platform

Final Implementation Report

Software Development Laboratory | 2025/2026

Course Unit: Software Development Laboratory
Academic Year: 2025/2026
Project Group: Group 20
Submission: Final delivery – May 2026
Defense: Week 15 – June 2026

Final implementation, deployment, and defense evidence

Abstract

This report presents the final implementation of the **Business Incubator Management Platform**, developed for the Software Development Laboratory course as the continuation of the Phase 1 analysis and design proposal. The original proposal identified a concrete operational problem: incubators manage resident companies, occupation contracts, financial obligations, rooms, equipment, and support requests through fragmented tools. The final system implements that proposal as a Docker-based microservices platform built primarily with Django REST Framework, PostgreSQL, RabbitMQ, Nginx, and a Next.js frontend.

The delivered product contains ten bounded backend services, each with its own runtime responsibility and database boundary, plus a gateway and frontend. It supports staff operations for company, contract, finance, space, booking, inventory, ticket, user, dashboard, and document management; client-company self-service workflows; and public booking requests. The implementation also includes asynchronous domain events, scheduler and consumer sidecars, seeded demo data, local verification evidence, a compiled defense report, a presentation slide deck, and a GitLab CI/CD path for changed-service deployment to the course production virtual machine.

The report follows the style and structure of the Phase 1 technical document: it revisits the product context and stakeholder requirements, maps the implemented system back to the original functional and non-functional requirements, documents the final architecture and service responsibilities, explains the main communication flows and deployment workflow, and records the verification and known operational limitations of the final delivery.

Keywords: incubator management, microservices, Django REST Framework, PostgreSQL, RabbitMQ, Docker Compose, GitLab CI/CD, Next.js, RBAC.

Contents

Abstract	i
1 Introduction and Project Continuity	1
1.1 Purpose of this Report	1
1.2 Original Problem Context	1
1.3 Final Delivery Scope	2
2 Requirements Baseline	4
2.1 Course Part 2 Technical Requirements	4
2.2 Phase 1 Product Requirements	5
2.3 Non-Functional Requirement Traceability	7
3 Final System Architecture	9
3.1 Architectural Pattern	9
3.2 Runtime Topology	9
3.3 Service Catalogue	10
3.4 Database-per-Service Design	11
3.5 Synchronous and Asynchronous Communication	12
4 Implemented Product Capabilities	13
4.1 Staff Back Office	13
4.1.1 Dashboard and Reporting	13
4.1.2 Company and Contract Administration	13
4.1.3 Space and Booking Operations	13
4.1.4 Inventory, Tickets, and Users	14
4.2 Client Portal	14
4.3 Public Booking Request	14
4.4 Document Management	14
5 Implementation Details	16
5.1 Repository Structure	16
5.2 Authentication and Authorization	16

5.3	Seed Data and Demo Readiness	17
5.4	Background Processing	17
5.5	Frontend Design and User Experience	17
6	Deployment and CI/CD	18
6.1	Local Runtime	18
6.2	Production Deployment Workflow	19
6.3	Remote Host Preparation	19
6.4	Environment Variables and Secrets	19
7	Verification and Evidence	20
7.1	Local Verification Strategy	20
7.2	Executed Checks	20
8	Limitations and Future Work	22
8.1	Known Limitations	22
8.2	Future Enhancements	22
9	Conclusion	24

List of Tables

2.1	Compliance with Part 2 technical requirements	4
2.2	Traceability from Phase 1 functional modules to implementation	6
2.3	Non-functional requirement traceability	7
3.1	Final backend service catalogue	10
3.2	Final domain event catalogue	12
7.1	Final verification evidence	20

Chapter 1

Introduction and Project Continuity

1.1 Purpose of this Report

The first phase of the project produced a formal product and technical proposal for a Business Incubator Management Platform. That document defined the problem context, stakeholder goals, functional and non-functional requirements, use cases, entity model, behavioral sequences, architecture, API catalogue, and user interface mock-ups. This final report has a different purpose: it documents the *implemented result*. It therefore treats the Phase 1 report as a baseline specification and describes how the delivered software satisfies the course infrastructure requirements and the original product requirements.

The report is also a defense artifact. It is intended to help evaluators inspect three dimensions of the delivery:

1. **Product completeness:** whether the implemented platform covers the core incubator workflows proposed in Phase 1.
2. **Technical compliance:** whether the implementation satisfies the Software Development Laboratory Part 2 requirements for microservices, databases, RabbitMQ, Docker, reverse proxy, authentication, and deployment.
3. **Operational readiness:** whether the repository contains enough scripts, documentation, seed data, and verification evidence to run, test, and defend the system repeatably.

1.2 Original Problem Context

The Phase 1 report framed the product around a recurring operational problem in business incubators. These organizations are neither ordinary property managers nor simple coworking operators. They must monitor resident companies as they mature,

maintain formal occupation contracts, calculate fees, manage physical spaces and equipment, answer support requests, and provide management reports to institutional stakeholders. When these activities are handled through spreadsheets, shared folders, and email, the resulting process becomes fragile: contract data may diverge from space occupancy data, payment information may not be visible to resident companies, and management indicators require manual aggregation.

The implemented platform addresses this problem by consolidating the operational data into a single system while preserving strict service boundaries. Staff users receive a management interface for the full incubator portfolio. Client users receive a portal scoped to their own company. Public users can submit booking requests without gaining access to private operational data.

1.3 Final Delivery Scope

The final repository delivers a production-shaped application stack with the following scope:

- Ten Django REST Framework backend services: authentication, company, contract, finance, space, booking, inventory, ticket, dashboard, and document.
- PostgreSQL persistence separated by service databases, matching the database-per-service design constraint.
- RabbitMQ asynchronous domain events with idempotent consumer handling.
- Nginx reverse proxy routing for frontend and API endpoints.
- Next.js and Ant Design frontend for staff, client, and public workflows.
- MinIO object storage for document payloads, with metadata managed by a document service.
- Docker Compose runtime with scheduler and consumer sidecars.
- GitLab CI/CD build, test, image push, and production deployment workflow.
- Defense artifacts: final report, presentation deck, runbooks, local QA evidence, and release checklist.

title=Final implementation summary

The final system is not just a prototype of isolated CRUD screens. It implements chained business flows across service boundaries: company creation leads to contracts and documents; contract activation affects spaces and

finance; bookings can reserve spaces and equipment; payments and tickets feed dashboard indicators; and domain events keep read models and side effects synchronized.

Chapter 2

Requirements Baseline

2.1 Course Part 2 Technical Requirements

The Software Development Laboratory Part 2 assignment required a microservices solution implemented with Django REST Framework, PostgreSQL, RabbitMQ, Docker, Nginx, authentication/authorization, GitLab CI/CD, and documentation. Table 2.1 maps those requirements to the final implementation.

Table 2.1: Compliance with Part 2 technical requirements

Requirement	Implemented evidence	Status
Minimum four microservices, with at least two core business services	Ten services are implemented. Company, contract, finance, space, booking, inventory, ticket, dashboard, document, and auth are separate Compose services.	Implemented
Django REST Framework API backend	Each bounded context is implemented as a Django/DRF service with serializers, viewsets, URLs, migrations, and management commands.	Implemented
Minimum eight entities/models	The final domain contains users, companies, employees, contracts, payments, spaces, space types, bookings, booking equipment, equipment types, equipment, tickets, ticket messages, documents, dashboard snapshots, and event-consumption records.	Implemented

Requirement	Implemented evidence	Status
PostgreSQL persistence	Compose defines service-specific PostgreSQL databases/users, initialized through database scripts and accessed through Django ORM.	Implemented
Authentication and authorization	JWT authentication is provided by the auth service; the gateway validates tokens and forwards trusted identity/company headers; services enforce role and company scoping.	Implemented
RabbitMQ asynchronous communication	RabbitMQ topic exchange <code>incubator.events</code> carries company, contract, booking, payment, and support events.	Implemented
Minimum eight documented events	The repository documents eleven events in <code>docs/events.md</code> .	Implemented
Nginx reverse proxy	Gateway routes <code>/api/auth/</code> , <code>/api/companies/</code> , <code>/api/contracts/</code> , etc., and serves the frontend route space.	Implemented
Docker Compose orchestration	<code>infra/docker-compose.yml</code> defines infrastructure, backend services, sidecars, frontend, and gateway.	Implemented
Separate configuration for environments	Development/runtime configuration is driven by environment files and Compose overlays, including a production image overlay.	Implemented
GitLab CI/CD with build, push, deployment stages	The final <code>.gitlab-ci.yml</code> validates, tests, builds changed images, pushes registry tags, and deploys through SSH and Docker Compose.	Implemented
Detailed README, report, and presentation material	README, architecture/deploy/event docs, this report, and the PowerPoint deck are included in the repository.	Implemented

2.2 Phase 1 Product Requirements

The Phase 1 functional catalogue was organized into ten product modules. The final implementation keeps the same domain decomposition. Table 2.2 provides a concise

traceability view from the original modules to the delivered functionality.

Table 2.2: Traceability from Phase 1 functional modules to implementation

Phase 1 module	Final implementation result	Status
Company registration	Company CRUD, employee records, maturity/CAE data, archive state, document links, seeded companies, and staff/client views.	Implemented
Company profiles and segmentation	Staff can inspect and edit company data; client portal exposes company-scoped profile data; dashboard includes company/maturity indicators.	Implemented
Contract management	Contract CRUD, lifecycle states, company/space association, fee fields, document support, expiration/termination commands, and contract events.	Implemented
Financial management	Payments, status tracking, revenue indicators, overdue processing, client payment history, staff finance page, and dashboard graphs.	Implemented
Space management	Space types, spaces, bookable flags, price configuration, occupancy/reservation state, contract/booking integration, and public availability.	Implemented
Booking system	Staff, client, and public booking flows; staff approval/rejection; reservation availability; optional equipment; price estimation and booking events.	Implemented
Inventory and equipment	Equipment types, equipment catalogue, optional space assignment, booking attribution history, rental price metadata, and booking event consumers.	Implemented
User management and RBAC	Director/staff/client roles, user CRUD, JWT sessions, company isolation through identity headers, and route guards.	Implemented
Support tickets	Client ticket creation, staff queue, messages, status transitions, assignment support, and dashboard ticket indicators.	Implemented

Phase 1 module	Final implementation result	Status
Dashboards and reporting	Staff dashboard and finance dashboard include charts, KPIs, revenue/status breakdowns, occupancy summaries, and operational health data.	Implemented

2.3 Non-Functional Requirement Traceability

The non-functional requirements from Phase 1 guided implementation decisions. Security and maintainability received the strongest emphasis because they affect all services. Table 2.3 summarizes the main quality attributes.

Table 2.3: Non-functional requirement traceability

Quality attribute	Implementation decision	Status
Security	JWT authentication, password hashing through Django, gateway verification, role-aware APIs, company scoping through X-Company-Id, and non-committed deployment secrets.	Implemented
Scalability	Service-per-domain Compose containers and database-per-service layout allow independent image builds and service restarts.	Implemented
Performance	Dashboard aggregation is delegated to a dashboard service; frontend uses API clients and selected charts; local tests validate expected API helpers.	Implemented
Reliability	RabbitMQ consumers store processed <code>event_id</code> values, scheduler sidecars replace in-process cron, and health checks are declared in Compose.	Implemented
Usability	Staff and client shells, dark theme, responsive tables, select-based forms, computed price estimates, and clearer dashboard/finance charts were implemented.	Implemented
Maintainability	Monorepo keeps shared infrastructure visible while preserving service-local Django projects, migrations, URLs, and tests.	Implemented

Quality attribute	Implementation decision	Status
Deployment	GitLab CI/CD builds changed images and deploys them through Docker Compose on the assigned VM.	Implemented

Chapter 3

Final System Architecture

3.1 Architectural Pattern

The final architecture follows the microservice design proposed in Phase 1. Each bounded context is implemented as a separate Django service, exposed through the Nginx gateway and deployed as an independent Docker image. The monorepo is used for course delivery and integration convenience, but the runtime model preserves service ownership boundaries.

title=Architecture rationale

The decision to use ten services instead of a smaller four-service design was intentional. The product domain has naturally separate responsibilities: identity, company data, contracts, finance, spaces, bookings, inventory, tickets, dashboard aggregation, and documents. Separating these areas makes the event flows explicit and supports the course requirement to demonstrate multiple core business services communicating through APIs and RabbitMQ.

3.2 Runtime Topology

The runtime topology is composed of four layers:

1. **Edge layer:** Nginx gateway and Next.js frontend.
2. **Application layer:** ten Django REST Framework services and service-specific scheduler/consumer sidecars.
3. **Messaging layer:** RabbitMQ topic exchange for asynchronous integration.
4. **Persistence layer:** PostgreSQL databases and MinIO object storage.

```

1 Browser
2   -> Nginx gateway
3     -> Next.js frontend
4     -> /api/auth/          -> auth-service          -> auth database
5     -> /api/companies/    -> company-service      -> company database
6     -> /api/contracts/    -> contract-service    -> contract
7                               database
8     -> /api/finance/      -> finance-service      -> finance database
9     -> /api/spaces/       -> space-service        -> space database
10    -> /api/bookings/     -> booking-service      -> booking database
11    -> /api/inventory/    -> inventory-service    -> inventory
12                               database
13    -> /api/tickets/      -> ticket-service       -> ticket database
14    -> /api/dashboard/    -> dashboard-service    -> dashboard
15                               database
16    -> /api/documents/    -> document-service     -> document
17                               database + MinIO
18
19 Django services <-> RabbitMQ topic exchange incubator.events

```

Listing 3.1: Simplified runtime topology

3.3 Service Catalogue

Table 3.1: Final backend service catalogue

Service	Responsibility	Gateway prefix	Port
auth-service	User accounts, JWT login, token verification, role data, and company identity for client users.	/api/auth/	8001
company-service	Company profiles, maturity stage, CAE classification, employees, archive status, and company events.	/api/companies/	8002
contract-service	Contracts, lifecycle transitions, assigned spaces, rates, monthly fee snapshots, expiration, and contract events.	/api/contracts/	8003

Service	Responsibility	Gateway prefix	Port
finance-service	Payments, billing records, overdue processing, revenue reports, and payment events.	/api/finance/	8004
space-service	Space types, spaces, occupancy, reservation blocking, booking availability, and space projections.	/api/spaces/	8005
booking-service	Staff/client/public booking requests, approvals, cancellations, quote fields, and booking events.	/api/bookings/	8006
inventory-service	Equipment types, equipment items, assignment history, booking-equipment allocation, and inventory consumers.	/api/inventory/	8007
ticket-service	Support tickets, messages, assignment/status workflow, and company-scoped ticket access.	/api/tickets/	8008
dashboard-service	Aggregated KPIs, service health, ticket metrics, and dashboard read models.	/api/dashboard/	8009
document-service	Document metadata, file upload/download/delete, and MinIO object storage integration.	/api/documents/	8010

3.4 Database-per-Service Design

The Compose stack creates separate PostgreSQL databases and users for the services. This directly implements the database-per-service pattern required by the assignment and proposed in Phase 1. The design avoids shared-table coupling: services exchange identifiers and selected denormalized fields through APIs or events rather than joining each other's tables.

The database separation is especially important for:

- **Contract and finance consistency:** contract rates are copied into billing/-payment contexts rather than recomputed from mutable company data.
- **Space and booking availability:** bookings reserve spaces for time windows without making the booking service own the full space catalogue.

- **Client isolation:** downstream services can enforce company scope using the trusted identity headers produced by the gateway.

3.5 Synchronous and Asynchronous Communication

The implementation uses synchronous REST calls for user-facing queries and commands that require immediate responses. It uses RabbitMQ events for cross-service side effects and eventually consistent read models.

Table 3.2: Final domain event catalogue

Event	Purpose	Documented
<code>company.created</code>	Notify downstream services that a company profile exists.	Yes
<code>company.archived</code>	Preserve historical data while signalling inactive company state.	Yes
<code>employee.changed</code>	Update employee-count indicators and company growth data.	Yes
<code>contract.activated</code>	Mark contract-dependent projections such as occupied spaces and finance records.	Yes
<code>contract.expired</code>	Release dependent resources when contract end date is reached.	Yes
<code>contract.terminated</code>	Release dependent resources and preserve termination reason.	Yes
<code>booking.approved</code>	Reserve a space/equipment time window and feed dashboards.	Yes
<code>booking.rejected</code>	Record staff decision for client/public visibility.	Yes
<code>booking.cancelled</code>	Release reservations and equipment allocations.	Yes
<code>booking.completed</code>	Mark the end of temporary resource usage.	Yes
<code>payment.recorded</code>	Update finance and dashboard indicators after payment.	Yes

Every event envelope includes an `event_id`. Consumers record processed IDs so retries do not duplicate side effects. This is an important reliability property for RabbitMQ because the broker may redeliver messages after network or consumer failures.

Chapter 4

Implemented Product Capabilities

4.1 Staff Back Office

The staff interface is the operational center of the platform. It exposes the main administrative domains from the Phase 1 sitemap: dashboard, companies, contracts, spaces, bookings, inventory, finance, tickets, and users. The UI was refined beyond placeholder pages into a consistent dark-themed management experience with navigation, tables, forms, charts, and contextual detail areas.

4.1.1 Dashboard and Reporting

The dashboard implements the reporting objectives from FR-1001 and FR-1002. It shows company totals, maturity/sector breakdowns, occupancy indicators, financial summaries, ticket metrics, and service health. The finance page adds charts for revenue and payment state so that staff can quickly understand the current operational picture without exporting data.

4.1.2 Company and Contract Administration

Company administration supports legal/profile fields, CAE and maturity data, employees, and document references. Contract administration supports company and space association, rate fields, lifecycle state, and document attachment. The contract lifecycle preserves the Phase 1 rule that active contract economic values are treated as historical snapshots rather than being silently changed by future company classification changes.

4.1.3 Space and Booking Operations

The space module supports space types, space prices, capacity, bookable flags, occupancy state, and reservation blocking. The booking module supports staff review

of client and public booking requests, approval/rejection workflows, and booking cost fields. Several UX corrections were made in the final stage: reservation-related forms use selects and contextual labels instead of raw UUID text inputs; pricing is computed from space/equipment and duration where appropriate; and the staff reservation table places detailed information in the expanded row instead of overloading columns.

4.1.4 Inventory, Tickets, and Users

Inventory management covers equipment types, equipment items, flexible rental cost metadata, optional assignment to a space, and attribution history. Ticket management supports client-created support requests, staff status handling, and message threads. User management supports director/staff/client roles, client company association, and deactivation/edit flows.

4.2 Client Portal

The client portal implements the Phase 1 requirement that resident companies should see their own information without accessing other companies' data. Client routes expose company profile information, contract details, payments, bookings, and support tickets. The gateway and API clients carry the company identity so downstream services can enforce **X-Company-Id** isolation.

The client booking flow was refined to improve usability: available spaces are loaded into select inputs, equipment options show price context, reserved time windows are blocked, and estimated price is calculated instead of being entered as a free-form number. This aligns the implemented product with the original usability requirements for clear form validation and low-error workflows.

4.3 Public Booking Request

The public booking route allows external users to request bookable spaces by providing contact and company information. The request enters the booking workflow as pending and requires staff review, satisfying the Phase 1 public booking requirement while preserving staff control over actual resource reservation.

4.4 Document Management

The document service manages metadata and MinIO object storage for company and contract documents. Upload, list, download, and delete flows are available through service endpoints and integrated with seeded demo data. This supports the original

requirement that signed contracts and supporting documents remain linked to the appropriate company or contract records.

Chapter 5

Implementation Details

5.1 Repository Structure

The repository is organized as a deployable monorepo. The most important paths are:

- `services/*-service/`: Django REST Framework services.
- `frontend/`: Next.js 14 frontend and API client modules.
- `gateway/`: Nginx routing configuration.
- `infra/docker-compose.yml`: main runtime stack.
- `infra/docker-compose.production.yml`: production image overlay.
- `infra/scripts/ci/`: changed-service detection, image build, and remote deploy scripts.
- `libs/py-common/`: shared Python utilities, including event envelope support.
- `docs/`: architecture, events, deployment, defense, and report artifacts.

5.2 Authentication and Authorization

Authentication uses email/password login with JWT issuance by the auth service. The gateway validates incoming tokens and forwards trusted headers to backend services. This keeps token verification consistent at the edge while allowing individual services to enforce domain-specific authorization rules. Staff and director users can access portfolio-wide data; client users are restricted to their associated company.

5.3 Seed Data and Demo Readiness

The final delivery includes seed data for the primary workflows: users, companies, contracts, payments, spaces, space types, bookings, inventory items, tickets, and documents. The seed set is designed for demonstration rather than for artificial benchmark volume. Its purpose is to ensure that each screen and business flow has linked data: companies have contracts, contracts have finance records, spaces and equipment have prices, clients have tickets/bookings, and documents can be downloaded.

5.4 Background Processing

The implementation intentionally avoids Celery, Kubernetes, and cloud services. Background work is performed through scheduler and consumer sidecars:

- Contract scheduler: detects expirations and publishes lifecycle events.
- Finance scheduler: generates/updates billing and overdue payment state.
- Booking scheduler: completes or releases time-bound reservations.
- Service consumers: process RabbitMQ events and update local projections.

This design is simpler than a full distributed job platform but sufficient for the course requirements and easier to demonstrate through Docker Compose.

5.5 Frontend Design and User Experience

The Phase 1 report contained mockups for staff and client screens. The final frontend follows the same navigation concept but adapts the visual design into a cohesive dark operational theme. Ant Design components provide tables, forms, selects, date/time inputs, and feedback messages. The final UI work focused on removing low-quality interactions such as raw UUID inputs, overly wide tables, misaligned dashboard cards, and missing price context.

The most important UX decisions were:

- Use entity labels and selects instead of requiring users to type identifiers.
- Keep staff tables readable by moving secondary details into expanded cards.
- Compute estimates from price and duration instead of accepting arbitrary values from client users.
- Keep the client portal visibly separated from staff administration.
- Make dashboard cards and charts visually consistent for defense use.

Chapter 6

Deployment and CI/CD

6.1 Local Runtime

The project can be started locally either through the repository Makefile or through Tilt. Both paths use the same Docker Compose topology: PostgreSQL, RabbitMQ, Redis, MinIO, backend services, scheduler/consumer sidecars, frontend, and gateway. Health checks and entrypoint scripts prepare migrations and service readiness.

The Makefile path is useful for a compact full-stack run and for final demo validation:

```
1 cp .env.example .env
2 make up
3 make seed
4 make demo
5 make logs
6 make down
```

Listing 6.1: Make-based local runtime commands

Tilt is useful during development because it watches service-specific folders and rebuilds or restarts only the affected resources instead of restarting the whole stack for unrelated file changes. This is especially important in the monorepo, where documentation, frontend, and backend files are edited frequently.

```
1 cp .env.example .env
2 tilt up
3 # open the Tilt web UI, inspect resource logs, and trigger
   targeted rebuilds
4 tilt down
```

Listing 6.2: Tilt-based local development commands

6.2 Production Deployment Workflow

The final CI/CD implementation uses GitLab stages for validation, testing, image build, and production deployment. Changed-service detection reduces the amount of work performed for each main-branch update. A service-local change builds and deploys the affected image and any associated sidecar service. Shared library, Compose, deployment, or environment-template changes trigger a full rollout.

```
1 # detect changed deployable services
2 bash infra/scripts/ci/detect-changed-services.sh
3
4 # build and push only affected images
5 bash infra/scripts/ci/build-and-push.sh
6
7 # deploy affected compose services remotely
8 bash infra/scripts/ci/deploy-remote.sh
```

Listing 6.3: Changed-service deployment shape

The production deployment target is the assigned course virtual machine. The user is fixed as `a69603`; the host and password are stored as protected GitLab variables. The repository does not contain the VM password, production environment file, or registry credentials.

6.3 Remote Host Preparation

The course VM was checked over SSH. Docker Engine and Docker Compose v2 were installed, the service user was added to the `docker` group, and the versions were verified. This prepares the VM for the GitLab deployment job once the pushed pipeline reaches the deploy stage.

6.4 Environment Variables and Secrets

The final deployment workflow distinguishes repository configuration from secret configuration. The repository contains `.env.example` and CI scripts, but runtime secrets are provided through GitLab protected variables or the remote shared `.env` file. This matches the Part 2 requirement to use environment variables for credentials and avoids committing secrets to the repository.

Chapter 7

Verification and Evidence

7.1 Local Verification Strategy

Because the project uses shared GitLab runners, local verification was treated as the primary merge gate. The goal was to prove correctness before relying on remote CI availability. Verification was split into host-only checks, frontend checks, Python checks, Compose checks, report/deck generation, SSH connectivity, and VM readiness.

7.2 Executed Checks

Table 7.1: Final verification evidence

Check	Purpose	Result
<code>bash -n infra/scripts/ci/*.sh</code>	Validate CI helper shell syntax.	Pass
GitLab YAML parse	Validate parent and child pipeline YAML files.	Pass
Changed-service detection smoke	Confirm correct deploy variable generation.	Pass
Docker Compose production config	Validate base Compose plus production image overlay.	Pass
<code>npm run lint</code>	Run root/frontend lint workflow.	Pass
<code>npm -prefix frontend run typecheck</code>	TypeScript validation.	Pass
<code>npm -prefix frontend test</code>	Frontend unit/API helper tests.	Pass
<code>npm -prefix frontend run build</code>	Production frontend build.	Pass
<code>python3 -m ruff check .</code>	Python style/static validation.	Pass
<code>pytest infra/tests</code>	Infrastructure/runtime wiring tests.	Pass

Check	Purpose	Result
<code>latexmk -pdf ...</code> <code>final-report.tex</code>	Compile this report.	Pass
PowerPoint ZIP/XML validation	Confirm deck is a valid PPTX package.	Pass
SSH smoke to course VM	Confirm VM login and host identity.	Pass
Docker/Compose VM check	Confirm remote Docker Engine and Compose availability.	Pass

Chapter 8

Limitations and Future Work

8.1 Known Limitations

The final implementation satisfies the main product and technical requirements, but several areas remain natural candidates for future improvement:

- **Full localization:** user-facing Portuguese copy is present in the UI, but a complete bilingual translation audit would be separate work.
- **Advanced analytics:** dashboard and finance charts cover the main defense indicators, while deeper historical reports could be extended with exportable filters and longer time-series analysis.
- **Hard production security:** the course deployment uses the assigned VM and protected CI variables. A real production installation should add TLS, backup automation, external monitoring, and a stricter secrets manager.
- **Performance benchmarking:** functional and build checks were run; formal load tests against hundreds of companies remain future work.

8.2 Future Enhancements

Future work should prioritize the following improvements:

1. Add formal OpenAPI publication for every service and link it from the README.
2. Add scheduled backups for PostgreSQL and MinIO on the production VM.
3. Extend dashboard reporting with date-range filters and downloadable CSV summaries.
4. Add end-to-end Playwright tests for the complete staff/client/public demo path once the production deployment is stable.

5. Introduce monitoring dashboards for RabbitMQ queues, service health, and failed background jobs.

Chapter 9

Conclusion

The final implementation converts the Phase 1 proposal into a functioning microservices platform. It preserves the original product intent—centralized, role-aware management of incubated companies, contracts, finance, spaces, bookings, inventory, tickets, documents, and dashboards—while meeting the technical constraints of Part 2: Django REST Framework services, PostgreSQL, RabbitMQ, Docker Compose, Nginx reverse proxy, authentication/authorization, GitLab CI/CD, and deployment automation.

The most important result is the integration between business workflows. The system does not treat domains as unrelated CRUD tables: contracts influence spaces and finance; bookings influence spaces and equipment; payments and tickets feed reporting; documents are attached to company and contract records; and client-company users are isolated from other companies' data. This is the core value promised by the original proposal and delivered in the final branch.

Together with the README, architecture documentation, deployment notes, event catalogue, seed data, presentation deck, and local verification evidence, this report provides the final technical handoff for evaluation and defense.